

ForestGuard Agentic AI – Product Requirements Document (PRD)

Project Name: ForestGuard

Version: 2.0 (Agentic Architecture)

Type: Autonomous AI Agent for Deforestation Detection & Reporting

1. Executive Summary

ForestGuard is an autonomous AI agent designed to monitor forests using freely available satellite imagery, detect potential illegal deforestation, verify the event using geospatial analysis, reason over contextual information, generate evidence-backed reports, and automatically notify the relevant authorities.

Unlike traditional monitoring systems that follow a fixed pipeline, ForestGuard adopts an **agentic architecture**, where a Large Language Model (LLM) dynamically decides which tools to use, in what order, and when sufficient evidence has been gathered before taking action.

The project supports:

- SDG 13 – Climate Action
 - SDG 15 – Life on Land
-

2. Vision

Create a fully autonomous environmental monitoring agent capable of continuously observing forests, reasoning about environmental changes, and assisting authorities in combating illegal logging without requiring human intervention.

3. Problem Statement

Current forest monitoring systems suffer from several limitations:

- Manual inspection of satellite alerts
- Large number of false positives
- Delayed report generation
- No contextual reasoning
- Lack of automated evidence collection
- High workload for conservation agencies

Although free satellite datasets already exist, there is no intelligent layer capable of transforming raw alerts into actionable intelligence.

ForestGuard fills this gap.

4. Objectives

The system should:

- Monitor forests continuously.
 - Detect possible deforestation.
 - Verify vegetation loss.
 - Check legal protection status.
 - Analyze historical incidents.
 - Estimate severity.
 - Generate evidence reports.
 - Notify authorities automatically.
 - Maintain historical memory for future reasoning.
-

5. Functional Requirements

Detection

- Retrieve latest GLAD/RADD alerts
 - Fetch Sentinel imagery
 - Calculate NDVI
 - Detect vegetation loss
-

Context Analysis

- Protected area lookup
 - Historical incident lookup
 - Permit verification (future)
 - Weather verification (future)
-

Reasoning

The AI should determine:

- Is this illegal logging?
 - Is evidence sufficient?
 - What is the confidence level?
 - Should additional verification be performed?
 - What action should be recommended?
-

Report Generation

Generate:

- PDF Report
- JSON Report

Containing:

- Coordinates
 - Area affected
 - Before/After imagery
 - NDVI values
 - Timestamp
 - Risk score
 - Evidence
 - Recommendation
-

Notification

Automatically send reports to:

- Forest Department
 - NGOs
 - Dashboard
-

6. Non Functional Requirements

- Fast response
- Modular architecture
- Easily extensible
- Fault tolerant

- API based
 - Secure
 - Observable
 - Scalable
-

7. Technology Stack

Backend

- FastAPI
- Python

Database

- PostgreSQL
- PostGIS

AI

- Claude API

Geospatial

- GeoPandas
- Rasterio
- Shapely

Computer Vision

- OpenCV
- NumPy

Frontend

- React

Reports

- ReportLab
- python-docx

Notifications

- SMTP
- SendGrid

Scheduling

- Cron
- Airflow

8. Existing Backend Architecture

Scheduler

↓

Fetch Alerts

↓

Fetch Satellite Images

↓

NDVI

↓

Protected Area

↓

Claude

↓

Generate Report

↓

Email

This is an intelligent automation pipeline.

9. Target Agentic Architecture

Trigger

↓

ForestGuard Agent

↓

Planner

↓

Tool Selection

↓

Execute Tool

↓

Evaluate

↓

Need More Evidence?

↓

Yes → Plan Again

↓

No

↓

Generate Report

↓

Notify Authority

↓

Store Memory

10. Agent Architecture

The AI Agent consists of five major components.

Planner

Responsible for deciding:

- What to do next
- Which tool to use
- Whether enough evidence exists

Tool Registry

Each backend service is converted into an independent tool.

Available Tools

GetForestAlertsTool

FetchSatelliteImageTool

ComputeNDVITool

ProtectedAreaLookupTool

HistoricalAlertTool

GenerateReportTool

NotificationTool

DashboardUpdateTool

DatabaseTool

Executor

Responsible for

- Executing tools
 - Capturing outputs
 - Returning results to planner
-

Memory

Short Term

Current alert

Coordinates

Images

NDVI

Temporary reasoning

Long Term

Historical alerts

Generated reports

Protected areas

Statistics

Authority mapping

Reasoning Engine

Claude performs:

- Planning
- Risk assessment
- Report drafting
- Recommendation generation
- Decision making

11. Agent Workflow

Step 1

Receive scheduled monitoring event.



Step 2

Planner inspects current context.



Step 3

Select GetForestAlertsTool.



Step 4

Retrieve latest alerts.



Step 5

Determine whether imagery is required.



Step 6

Call FetchSatelliteImageTool.



Step 7

Run NDVI verification.



Step 8

Call ProtectedAreaLookupTool.



Step 9

Retrieve historical alerts.



Step 10

Reason over all collected evidence.



Step 11

If evidence insufficient

Return to planner.



Otherwise



Generate Report



Notify Authorities



Store Report



Finish

12. Proposed Backend Structure

```
backend/  
  
api/  
  
services/  
  
database/  
  
models/  
  
utils/  
  
agent/  
  
planner.py  
  
executor.py  
  
tools.py  
  
memory.py  
  
prompts.py  
  
agent.py  
  
reporting/  
  
notifications/  
  
main.py
```

13. Planner Responsibilities

Planner should decide:

Which tool should execute next?

Should more evidence be collected?

Is NDVI verification necessary?

Should another satellite image be requested?

Should historical data be consulted?

Can report generation begin?

Should authorities be notified?

14. Executor Responsibilities

Execute tool

Capture output

Update context

Return updated state

Repeat until planner signals completion

15. Tool Registry

Each backend module becomes an AI Tool.

Tool	Purpose
GetForestAlertsTool	Retrieve GFW alerts
FetchSatelliteImageTool	Download Sentinel imagery
ComputeNDVITool	Vegetation verification
ProtectedAreaLookupTool	Protected region lookup
HistoricalAlertTool	Previous incidents
ReportGeneratorTool	PDF generation
NotificationTool	Email authority
DashboardTool	Dashboard update
DatabaseTool	Database operations

16. Memory Design

Short-Term Memory

Current execution context

Latest alerts

Downloaded imagery

Intermediate reasoning

Temporary observations

Long-Term Memory

Previous reports

Historical alerts

Protected area metadata

Authority contacts

Statistics

Future RAG knowledge base

17. Prompt Design

System Prompt

"You are ForestGuard, an autonomous environmental monitoring AI Agent.

Your goal is to investigate possible illegal deforestation.

Always gather sufficient evidence before making conclusions.

Use available tools when additional information is required.

Never generate reports without verification.

When enough evidence has been collected, produce a structured report and recommend appropriate actions."

18. Future RAG Integration

Future versions may include:

Environmental laws

Government regulations

Forest permits

Conservation policies

Scientific papers

Previous investigation reports

The planner will retrieve these documents before reasoning.

19. Multi-Agent Architecture (Future)

Supervisor Agent

↓

Detection Agent

↓

Vision Agent

↓

Verification Agent

↓

Risk Assessment Agent

↓

Reporting Agent

↓

Notification Agent

This architecture allows different specialized agents to collaborate.

20. API Flow

Scheduler

↓

POST /monitor

↓

Planner

↓

Tool Calls

↓

Reasoning

↓

Generate Report

↓

POST /notify

↓

Dashboard Update

21. Development Roadmap

Phase 1 (Completed)

- Backend APIs
 - Database
 - Authentication
 - Satellite integration
 - NDVI verification
 - Claude integration
 - Report generation
 - Notifications
-

Phase 2 (Current)

- Agent Planner
 - Tool Registry
 - Executor
 - Agent Memory
 - Prompt Engineering
-

Phase 3

- Tool Calling
 - Conversation Interface
 - Dynamic Planning
 - Human Approval Mode
-

Phase 4

- RAG Integration
 - Multi-Agent Collaboration
 - Long-Term Memory
 - Autonomous Learning
 - Distributed Deployment
-

22. Success Metrics

Technical

- Detection accuracy

- False positive rate
- NDVI precision
- Planner success rate
- Tool execution latency

Operational

- Time to generate report
- Notification success
- Area monitored
- Reports generated
- Authority response rate

23. Future Enhancements

- LangGraph-based orchestration
- OpenAI Agents SDK support
- Multi-agent collaboration
- Drone imagery integration
- SAR (Sentinel-1) support
- Citizen reporting portal
- Mobile application
- Live monitoring dashboard
- Voice interaction
- Natural language querying
- Predictive illegal logging analysis
- Carbon emission estimation
- Blockchain evidence storage

24. Conclusion

ForestGuard is evolving from an AI-assisted automation pipeline into a fully autonomous environmental AI agent. The existing backend already provides the core capabilities—data ingestion, geospatial verification, report generation, and notification. The next phase introduces an agentic orchestration layer consisting of a Planner, Tool Registry, Executor, and Memory, enabling the LLM to dynamically select tools, gather additional evidence when required, and make informed decisions before acting. This architecture transforms ForestGuard into a scalable, modular, and production-ready AI agent suitable for research, hackathons, and real-world environmental monitoring deployments.